GRUPO 5 · REQUERIMIENTOS FUNCIONALES

Ahorros y Depósitos

Módulo: Ahorro programado, depósitos a plazo, tasas, plazos y vencimientos

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 5
Versión	1.0
Fecha	10 de octubre de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST, `/api/v2`) y aiquaa-sandbox-web (front, `/v2`)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training_v2
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre dos productos de inversión. El **ahorro programado**: un plan con una meta y un aporte mensual, que acumula saldo con cada aporte debitado de una cuenta y se completa al alcanzar la meta. Y el **depósito a plazo fijo**: un importe que se inmoviliza desde una cuenta por un plazo y una tasa pactados, con su interés proyectado, y que puede cancelarse anticipadamente cobrando el interés proporcional a los días transcurridos.

Fuera de alcance: El débito automático mensual de los aportes (cada aporte se dispara a mano); el retiro o rescate del dinero de un plan de ahorro; el pago de intereses sobre el ahorro (su tasa es informativa); el vencimiento automático de un depósito y su renovación; la capitalización de intereses (todo el interés es simple); penalidades por cancelación anticipada; e impuestos.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con la API key de su grupo (o con un token JWT de grupo) y automatiza escenarios BDD sobre el módulo de su grupo.
Cliente del banco (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es titular de cuentas, tarjetas, préstamos, beneficiarios, planes de ahorro y depósitos.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo sobre el schema `qa_training_v2`, bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz tipo home banking que consume la API a través del proxy `/api/proxy/v2/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v2/ahorros	RF-G5-01	Lista planes de ahorro, por titular y/o estado.
POST /api/v2/ahorros	RF-G5-02	Crea un plan de ahorro programado.
GET /api/v2/ahorros/{id}	RF-G5-03	Devuelve el detalle de un plan con lo que falta para la meta.
PUT /api/v2/ahorros/{id}	RF-G5-04	Reemplaza las condiciones de un plan activo.
DELETE /api/v2/ahorros/{id}	RF-G5-05	Da de baja un plan de ahorro.
POST /api/v2/ahorros/{id}/aportar	RF-G5-06	Aporta al plan debitando la cuenta asociada.
GET /api/v2/depositos	RF-G5-07	Lista depósitos a plazo, por titular y/o estado.
POST /api/v2/depositos	RF-G5-08	Constituye un depósito a plazo debitando una cuenta.
GET /api/v2/depositos/{id}	RF-G5-09	Devuelve el detalle de un depósito con su interés proyectado.
POST /api/v2/depositos/{id}/cancelar	RF-G5-10	Cancela anticipadamente y acredita capital más interés.
POST /api/v2/g5/auth/token	RF-G5-TK	Emite el token JWT del grupo.

2. Precondiciones y acceso

- **Base URL de la API:** `https://aiquaa-sandbox-api.vercel.app`. La documentación interactiva del curso está en `/docs/v2` y la especificación OpenAPI en `GET /api/v2/docs`. La aplicación web está en `https://aiquaa-sandbox-web.vercel.app`.
- Toda request debe autenticarse por una de dos vías: el header `x-api-key` con una clave activa de `public.api_keys`, o el header `Authorization: Bearer <token>` con un JWT de grupo vigente. Si se envían ambos, manda `x-api-key`. Sin credenciales, o con credenciales inválidas, inactivas o vencidas, la respuesta es `401 UNAUTHORIZED` con un mensaje genérico que no indica cuál fue el motivo.
- Las rutas bajo `/api/v2` solo aceptan credenciales del **curso 2**: una API key válida pero del curso 1 recibe `403 FORBIDDEN` con el mensaje "Esta API key pertenece al curso 1 y esta ruta es del curso 2.". El gate es simétrico: una key del curso 2 tampoco sirve en `/api/v1`.
- Cada credencial admite un máximo de **30 requests por minuto** (ventana deslizante, contada por API key o por usuario del token, no por IP). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training_v2` (10 tablas) y se cargan con `scripts/seed-data-v2.sql`: 8 clientes (más un cliente por grupo, usado por el login de token), 12 cuentas, 20 movimientos, 10 tarjetas, 6 préstamos con sus cuotas, 10 beneficiarios, 12 transferencias, 6 planes de ahorro y 6 depósitos. La data inicial es determinística, pero **el sandbox es compartido por los cinco grupos**: los registros sembrados pueden haber sido modificados por otra suite. Un escenario robusto crea sus propios datos en lugar de depender del estado de una fila sembrada.
- **Clientes de soporte:** `GET|POST /api/v2/usuarios` y `GET|PUT|DELETE /api/v2/usuarios/{id}` no pertenecen a ningún grupo y están disponibles para todos; sirven para crear el titular de los productos de cada escenario. `GET /usuarios` acepta el filtro `email`; `POST` exige `nombre`, `email` y `documentoNumero` (email y documento son únicos: repetirlos da `409 CONFLICT`).
- **Sandbox SQL:** `POST /api/v2/sql/select` y `POST /api/v2/sql/update` reciben `{ "sql", "params" }` y permiten consultar o preparar datos directamente sobre las 10 tablas del curso. Admiten un único statement por request, con placeholders `$1..$n`; el `UPDATE` exige cláusula `WHERE`. Son la vía para verificar en base lo que hizo un endpoint, o para forzar un estado que la API no produce.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` hace baja lógica (`activo = false`, o `activa = false` en `cuentas`) y devuelve `204 No Content` sin cuerpo. Una fila dada de baja deja de verse en `GET`, `PUT` y `DELETE`, pero sigue existiendo en la base.
- En la aplicación web el acceso tiene tres capas, en orden: elegir el curso (`/curso`, opción 2), cargar la API key del curso (`/login`, que valida que la key sea del curso elegido) y elegir el cliente con el que se opera (`/auth/login`, por email de un cliente activo). Las pantallas `/v2/usuarios/*` son la excepción a la tercera capa: se pueden usar solo con la API key, porque son el punto de partida para crear un cliente.
- Ambos productos se alimentan de una cuenta: para aportar o constituir un depósito se necesita una cuenta `activa` con saldo suficiente. Una cuenta recién abierta nace en cero: hay que fondearla con `POST /api/v2/cuentas/{id}/movimientos` (tipo `credito`).
- Los endpoints de cuentas (`/api/v2/cuentas/**`) pertenecen al Grupo 1, pero este módulo los usa para preparar datos y para verificar saldos y movimientos después de cada operación.

- Datos sembrados: planes de ahorro 1 a 4 en `activo`, el 5 en `completado` y el 6 en `cancelado`; depósitos 1 a 3 en `activo`, 4 y 5 en `vencido` y el 6 en `cancelado`.
- **El resultado de cancelar un depósito depende de la fecha de ejecución**: el interés se prorratea por los días transcurridos desde su inicio. Un depósito creado y cancelado el mismo día genera interés cero. Para probar un interés mayor que cero hay que usar un depósito sembrado (con fecha de inicio pasada) o preparar la `fecha_inicio` por el sandbox SQL, y calcular el valor esperado a partir de la fecha del día.

3. Modelo de datos del módulo

ahorros

Planes de ahorro programado. `estado` es el estado de negocio y `activo` la marca de baja lógica.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`cuenta_id`	`bigint`, obligatorio, referencia a `cuentas(id)`: cuenta de la que se debitan los aportes
`nombre_meta`	`text`, obligatorio
`meta_monto`	`numeric(14,2)`, obligatorio, mayor que cero
`aporte_mensual`	`numeric(14,2)`, obligatorio, mayor que cero
`saldo_acumulado`	`numeric(14,2)`, obligatorio, por defecto 0; mayor o igual a cero
`tasa_anual`	`numeric(5,2)`, obligatoria, por defecto 0; mayor o igual a cero. Informativa
`estado`	`text`, uno de `activo` / `completado` / `cancelado`; por defecto `activo`
`activo`	`boolean`, obligatorio, por defecto `true`; `false` = dado de baja
`created_at`	`timestamptz`, por defecto `now()`

depositos

Depósitos a plazo fijo. Sus condiciones son inmutables una vez constituido.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`cuenta_id`	`bigint`, obligatorio, referencia a `cuentas(id)`: cuenta de la que sale y a la que vuelve el dinero
`monto`	`numeric(14,2)`, obligatorio, mayor que cero: el capital
`tasa_anual`	`numeric(5,2)`, obligatoria, mayor o igual a cero
`plazo_dias`	`integer`, obligatorio, entre 30 y 1095
`fecha_inicio`	`date`, obligatoria, por defecto la fecha del día
`fecha_vencimiento`	`date`, obligatoria, posterior a la fecha de inicio
`interes_generado`	`numeric(14,2)`, obligatorio, por defecto 0; se completa al cancelar
`estado`	`text`, uno de `activo` / `vencido` / `cancelado`; por defecto `activo`
`activo`	`boolean`, por defecto `true`; ningún endpoint lo cambia
`created_at`	`timestamptz`, por defecto `now()`

Campos calculados que la API agrega a las respuestas y que no existen como columnas: en un plan de ahorro, `falta_para_meta` = meta − saldo acumulado (negativo si se superó la meta); en un depósito, `interes_proyectado` = monto × tasa / 100 × plazo en días / 365, redondeado a dos decimales, y `dias_restantes` = fecha de vencimiento − fecha del día (negativo si ya pasó). Todo el interés es simple, con base de 365 días. Ejemplo: 5.000.000 al 11,5 % anual por 180 días proyecta un interés de 283.561,64.

4. Requerimientos funcionales

RF-G5-01 — Listar planes de ahorro

`GET /api/v2/ahorros`

Devuelve los planes de ahorro vigentes, con la posibilidad de acotar por titular y por estado.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `activo`, `completado` o `cancelado`.

Reglas de negocio

- Devuelve únicamente planes no dados de baja (`activo = true`), en cualquier estado de negocio.
- Los dos filtros se combinan entre sí.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de planes; cada elemento incluye las columnas de la tabla más el campo calculado `falta_para_meta`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` no es un entero positivo, o `estado` no es uno de los tres valores permitidos.

Fuente: `app/api/v2/ahorros/route.ts`

RF-G5-02 — Crear un plan de ahorro

`POST /api/v2/ahorros`

Crea un plan de ahorro programado con una meta y un aporte mensual de referencia. El plan arranca sin saldo.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `cuentaId` (obligatorio): número entero positivo de la cuenta de la que se debitarán los aportes.
- `nombreMeta` (obligatorio): texto no vacío.
- `metaMonto` (obligatorio): número mayor que cero, como número JSON.
- `aporteMensual` (obligatorio): número mayor que cero, como número JSON.
- `tasaAnual` (opcional): número entre 0 y 999,99; si se omite se guarda 0.

Reglas de negocio

- El plan nace con `estado = 'activo'` y `saldo_acumulado = 0`. **Crear el plan no debita la cuenta**: el saldo solo crece con aportes (RF-G5-06).
- El titular y la cuenta deben existir; un id inexistente es rechazado por la clave foránea.
- **No se valida que la cuenta pertenezca al titular, ni su estado ni su moneda**: esas condiciones recién se verifican al aportar.
- **No se valida que el aporte mensual sea menor o igual a la meta**, ni un mínimo de meta, ni un plazo máximo.
- La tasa anual se guarda pero no produce ningún efecto: el sandbox no liquida intereses sobre el ahorro.

Respuesta esperada

- `201 Created` con `data` conteniendo el plan creado, incluido `falta_para_meta` (igual a la meta).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `nombreMeta` está vacío, la meta o el aporte no son números positivos, la tasa está fuera de rango, o el titular o la cuenta no existen.

Fuente: `app/api/v2/ahorros/route.ts`

RF-G5-03 — Consultar un plan de ahorro

`GET /api/v2/ahorros/{id}`

Devuelve el detalle de un plan: meta, aporte, saldo acumulado, lo que falta y estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Si no existe o está dado de baja, responde 404 con el mensaje "Plan de ahorro no encontrado.".

Respuesta esperada

- `200 OK` con `data` conteniendo el plan completo y `falta_para_meta`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un plan vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/ahorros/[id]/route.ts`

RF-G5-04 — Modificar un plan de ahorro

`PUT /api/v2/ahorros/{id}`

Reemplaza nombre, meta, aporte mensual y tasa de un plan que sigue activo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `nombreMeta` (obligatorio): texto no vacío.
- `metaMonto` y `aporteMensual` (obligatorios): números mayores que cero.
- `tasaAnual` (opcional): número entre 0 y 999,99.

Reglas de negocio

- Solo se modifica un plan en estado `activo`. En otro estado responde 409 con el mensaje "Un plan 'completado' ya no puede modificarse." (con el estado que corresponda).
- La nueva meta no puede ser menor al saldo ya acumulado: responde 409 con el mensaje "La meta no puede ser menor al saldo ya acumulado.".
- Es un reemplazo: **omitir `tasaAnual` la vuelve a 0**, aunque antes tuviera otro valor.
- El saldo acumulado, el estado, el titular y la cuenta no forman parte del cuerpo; si se envían, se ignoran.
- **Bajar la meta hasta igualar el saldo acumulado no completa el plan**: el estado solo cambia a `completado` con un aporte.

Respuesta esperada

- `200 OK` con `data` conteniendo el plan ya actualizado y `falta_para_meta` recalculado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un plan vigente con ese identificador.
`CONFLICT`	409	El plan no está activo, o la nueva meta es menor al saldo acumulado.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o alguno está fuera de rango.

Fuente: `app/api/v2/ahorros/[id]/route.ts`

RF-G5-05 — Dar de baja un plan de ahorro

`DELETE /api/v2/ahorros/{id}`

Da de baja lógica un plan de ahorro, quitándolo de los listados.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false` sin ninguna validación previa: no mira el estado ni el saldo acumulado.
- **El dinero acumulado no se devuelve a la cuenta**: un plan con saldo se puede dar de baja y ese saldo queda inaccesible.
- La baja no cambia el `estado` de negocio del plan.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un plan vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/ahorros/[id]/route.ts`

RF-G5-06 — Aportar a un plan de ahorro

`POST /api/v2/ahorros/{id}/aportar`

Registra un aporte: debita la cuenta asociada, suma al saldo acumulado del plan y, si se alcanza la meta, deja el plan completado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `monto` (opcional, en el cuerpo): número mayor que cero, como número JSON. Si se omite —o la request no lleva cuerpo— se aporta el `aporte_mensual` del plan.

Reglas de negocio

- El plan debe estar en estado `activo`. En otro estado responde 409 con el mensaje "El plan está 'completado': no admite aportes." (con el estado que corresponda).
- La cuenta asociada debe existir y no estar dada de baja (si no, 404 "Cuenta asociada al plan no encontrada.") y estar `activa` (si no, 409 "La cuenta asociada está bloqueada: no admite débitos.").
- El saldo de la cuenta debe alcanzar para el aporte: si no, 409 "Saldo insuficiente en la cuenta para el aporte.".
- Si todo es válido: la cuenta baja en el importe, el saldo acumulado sube en el mismo importe, y se registra en la cuenta un movimiento `debito` con `referencia_tipo = 'ahorro'`, `referencia_id` igual al id del plan y descripción "Aporte a meta " seguido del nombre.
- Cuando el saldo acumulado alcanza o supera la meta, el plan pasa a `completado` y deja de admitir aportes y cambios.
- **El aporte puede superar lo que falta**: no se recorta al tope de la meta, y `falta_para_meta` queda negativo.
- No hay control de frecuencia: se pueden hacer varios aportes seguidos el mismo día.
- La operación es atómica: cuenta y plan se mueven juntos o no se mueven.

Respuesta esperada

- `201 Created` con `data` conteniendo el plan actualizado (con `falta_para_meta`) y, anidada en `data.cuenta`, la cuenta con su saldo ya debitado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe el plan, o su cuenta asociada fue dada de baja.
`CONFLICT`	409	El plan no está activo, la cuenta no está activa, o el saldo de la cuenta no alcanza.
`VALIDATION_ERROR`	400	El `monto` enviado es cero, negativo o no es un número JSON.

Fuente: `app/api/v2/ahorros/[id]/aportar/route.ts`

RF-G5-07 — Listar depósitos a plazo

`GET /api/v2/depositos`

Devuelve los depósitos a plazo del sandbox, con la posibilidad de acotar por titular y por estado.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `activo`, `vencido` o `cancelado`.

Reglas de negocio

- Los dos filtros se combinan entre sí.
- El filtro `estado=vencido` usa el estado guardado, no la fecha: un depósito `activo` con fecha de vencimiento pasada no aparece como vencido.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de depósitos; cada elemento incluye las columnas de la tabla más los campos calculados `interes_proyectado` y `dias_restantes`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` no es un entero positivo, o `estado` no es uno de los tres valores permitidos.

Fuente: `app/api/v2/depositos/route.ts`

RF-G5-08 — Constituir un depósito a plazo

`POST /api/v2/depositos`

Constituye un depósito a plazo fijo: debita el capital de una cuenta y lo deja inmovilizado hasta el vencimiento o hasta su cancelación anticipada.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `cuentaId` (obligatorio): número entero positivo de la cuenta de la que sale el capital.
- `monto` (obligatorio): número mayor que cero, como número JSON.
- `tasaAnual` (obligatorio): número entre 0 y 999,99, como número JSON. Una tasa de 0 es válida.
- `plazoDias` (obligatorio): número entero entre 30 y 1095, como número JSON.

Reglas de negocio

- La cuenta debe existir y no estar dada de baja (si no, 404 "Cuenta no encontrada.") y estar `activa` (si no, 409 "La cuenta está bloqueada: no admite débitos.").
- El saldo de la cuenta debe alcanzar: si no, 409 "Saldo insuficiente para constituir el depósito.". Depositar exactamente el saldo es válido.
- Si todo es válido: la cuenta baja en el importe, se crea el depósito con `estado = 'activo'`, `fecha_inicio` igual a la fecha del día, `fecha_vencimiento` igual a la fecha del día más el plazo, e `interes_generado = 0`.
- Se registra en la cuenta un movimiento `debito` con `referencia_tipo = 'deposito'`, `referencia_id` igual al id del depósito y descripción "Constitución de depósito a plazo".
- El interés proyectado es interés simple con base 365: monto × tasa / 100 × plazo / 365.
- **No se valida que la cuenta pertenezca al titular indicado**, ni hay monto mínimo de negocio. El depósito no guarda moneda: toma implícitamente la de la cuenta.
- La operación es atómica: cuenta y depósito se crean juntos o no se crea nada.

Respuesta esperada

- `201 Created` con `data` conteniendo el depósito creado (con `interes_proyectado` y `dias_restantes`) y, anidada en `data.cuenta`, la cuenta con su saldo ya debitado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, el monto no es positivo, la tasa está fuera de rango, el plazo no es un entero entre 30 y 1095, algún valor numérico llegó como texto, o el titular no existe.
`NOT_FOUND`	404	No existe una cuenta vigente con ese identificador.
`CONFLICT`	409	La cuenta no está activa, o su saldo no alcanza.

Fuente: `app/api/v2/depositos/route.ts`

RF-G5-09 — Consultar un depósito a plazo

`GET /api/v2/depositos/{id}`

Devuelve el detalle de un depósito: capital, tasa, plazo, fechas, interés proyectado, días restantes y estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Si no existe, responde 404 con el mensaje "Depósito no encontrado.".
- `dias_restantes` se recalcula en cada consulta contra la fecha del día, y puede ser negativo.
- `interes_proyectado` es siempre el del plazo completo, también en un depósito ya cancelado; lo efectivamente pagado está en `interes_generado`.
- Un depósito no se puede modificar ni dar de baja: `PUT` y `DELETE` sobre esta ruta responden 405. Para deshacerlo existe la cancelación (RF-G5-10).

Respuesta esperada

- `200 OK` con `data` conteniendo el depósito completo, `interes_proyectado` y `dias_restantes`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un depósito con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/depositos/[id]/route.ts`

RF-G5-10 — Cancelar anticipadamente un depósito

`POST /api/v2/depositos/{id}/cancelar`

Precancela un depósito activo: acredita en la cuenta el capital más el interés proporcional a los días efectivamente transcurridos, y deja el depósito cancelado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo. No lleva cuerpo.

Reglas de negocio

- Solo se cancela un depósito en estado `activo`. En otro estado responde 409 con el mensaje "El depósito ya está 'cancelado'." (con el estado que corresponda).
- Los días transcurridos son la fecha del día menos la fecha de inicio, con un mínimo de 0 y un máximo igual al plazo pactado.
- El interés prorrateado es monto × tasa / 100 × días transcurridos / 365, redondeado a dos decimales. No hay penalidad por cancelar antes.
- Un depósito cancelado el mismo día en que se constituyó genera interés 0 y devuelve exactamente el capital.
- Un depósito `activo` cuya fecha de vencimiento ya pasó se cancela cobrando el interés del plazo completo, nunca más.
- La cuenta asociada recibe capital más interés; se registra en ella un movimiento `credito` por ese total, con `referencia_tipo = 'deposito'` y descripción "Cancelación anticipada de depósito".
- El depósito queda con `estado = 'cancelado'` e `interes_generado` igual al interés pagado. Cancelar es definitivo.
- Si la cuenta asociada fue dada de baja, responde 404 con el mensaje "Cuenta asociada al depósito no encontrada." y el depósito sigue activo.
- **No se valida el estado de la cuenta**: el dinero se acredita aunque la cuenta esté `bloqueada` o `cerrada`.

Respuesta esperada

- `200 OK` con `data` conteniendo el depósito cancelado, el campo `dias_transcurridos` y, anidada en `data.cuenta`, la cuenta con su saldo ya acreditado. Esta respuesta no incluye `interes_proyectado` ni `dias_restantes`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe el depósito, o su cuenta asociada fue dada de baja.
`CONFLICT`	409	El depósito no está activo.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/depositos/[id]/cancelar/route.ts`

RF-G5-TK — Obtener el token del grupo

`POST /api/v2/g5/auth/token`

Intercambia el usuario y la contraseña del grupo por un token JWT que puede usarse en lugar de la API key en cualquier ruta del curso. La ruta en sí también exige estar autenticado (API key del curso 2 o un token vigente).

Entradas y validaciones

- `username` (obligatorio): texto de 1 a 64 caracteres. El usuario del grupo es `c2_g05_ahorros`.
- `password` (obligatorio): texto de 1 a 200 caracteres. Se entrega por el canal del curso; no se publica en este documento.

Reglas de negocio

- La contraseña se compara en la base de datos contra un hash bcrypt; nunca se devuelve ni se registra en claro.
- Usuario inexistente, contraseña incorrecta y credencial inactiva devuelven el mismo `400 VALIDATION_ERROR` con el mensaje "Usuario o password inválidos.": la respuesta no revela cuál de los tres falló. Es `400` y no `401` a propósito, para no confundirlo con el fallo de la API key.
- Si las credenciales son válidas pero pertenecen a otro grupo, la respuesta es `403 FORBIDDEN` con el mensaje "Estas credenciales pertenecen al grupo 1 y esta ruta es del grupo 5.".
- El token es un JWT firmado con HS256, emitido por `aiquaa-sandbox`, con vigencia de **3600 segundos**. Sus claims (`sub`, `usuarioId`, `grupo`, `curso`) son legibles: el token está firmado, no cifrado.
- El token identifica al grupo pero no restringe por grupo: con él se puede operar sobre los recursos de cualquier módulo del curso 2.
- La emisión no deja rastro en tablas de dominio (el curso 2 no tiene tabla de sesiones); solo queda la fila de auditoría.

Respuesta esperada

- `200 OK` con `data` conteniendo `token`, `tokenType` (`Bearer`), `expiresIn` (`3600`), `grupo` (`5`), `grupoNombre`, `curso` (`2`) y `usuario` (`id`, `nombre`, `email`, `username`).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `username` o `password`, están vacíos, o la combinación no es válida.
`FORBIDDEN`	403	Las credenciales son de otro grupo, o la API key usada para llamar es del curso 1.
`UNAUTHORIZED`	401	La request no trae API key ni token.

Fuente: `app/api/v2/g5/auth/token/route.ts`, `lib/group-token-route.ts` y `lib/jwt.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de cohorte (curso) → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Tipos en la respuesta:** los identificadores (`id`, `usuario_id`, `cuenta_id`, ...) viajan como **número**; los importes y tasas (`saldo`, `monto`, `limite_credito`, `tasa_anual`, ...) viajan como **texto con dos decimales** (por ejemplo `"5000000.00"`), por ser valores numéricos exactos de la base; las fechas sin hora viajan como `YYYY-MM-DD` y las marcas de tiempo (`created_at`, `fecha_pago`) en formato ISO 8601. Una aserción sobre un importe debe comparar contra texto o convertir a número antes.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo). Los campos no declarados en el esquema se ignoran sin error.
- **Coerción de tipos:** los identificadores (`id`, `usuarioId`, `cuentaId`, ...) se convierten de texto a número automáticamente, vengan por query, por ruta o en el cuerpo. Los **importes, tasas y plazos del cuerpo JSON deben enviarse como número**: enviarlos como texto (`"monto": "1000"`) devuelve `400 VALIDATION_ERROR`.
- **Cuerpo JSON:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body."; un cuerpo vacío se interpreta como `{}`. Un fallo del esquema devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid request." y el detalle del campo en `error.details`.
- **Reglas de negocio → 409:** cuando el cuerpo es válido pero el estado actual del recurso no permite la operación (saldo insuficiente, recurso en un estado que no admite la acción), la respuesta es `409 CONFLICT` con un mensaje en español que describe la regla. `400` queda para entradas inválidas.
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`, por ejemplo un `usuarioId` inexistente) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos (una fecha imposible, un número fuera de rango) cae en `400 EXECUTION_ERROR`. En los tres casos el `message` es el texto crudo de Postgres.
- **Importes con más de dos decimales:** la API no valida la cantidad de decimales; la base redondea a dos al guardar.
- **Operaciones que mueven dinero:** se ejecutan en una transacción con bloqueo de las filas involucradas; si alguna validación falla, no queda ningún cambio parcial (ni saldo modificado ni movimiento registrado).
- **Listados:** devuelven como máximo 100 filas ordenadas por identificador ascendente, sin paginación ni indicación de que hubo recorte. Los filtros opcionales se combinan entre sí (`AND`).
- **Headers de caudal:** toda respuesta que pasó el control de caudal incluye `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` (este último en milisegundos Unix).
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera el framework y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** las requests quedan registradas en `public.sql_audit_log` con la credencial usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Sin control de titularidad:** la API no tiene roles ni noción de "mis" productos. Cualquier credencial válida del curso puede leer y modificar los recursos de cualquier cliente, y ningún endpoint verifica que la cuenta indicada pertenezca al `usuarioId` indicado.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso, y las acciones que cambian un estado sin crear un registro (bloquear, activar, aprobar, pagar una cuota, anular, cancelar).
`201 Created`	POST que crea un recurso o registra un movimiento de dinero nuevo.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Faltan credenciales, la API key no existe o está inactiva, o el token es inválido o venció.
`FORBIDDEN`	403	La credencial es válida pero es de otro curso, o las credenciales de un grupo se usaron en la ruta de token de otro grupo.
`RATE_LIMITED`	429	Se superaron las 30 requests por minuto de esa credencial.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe o ya está dado de baja.
`CONFLICT`	409	Regla de negocio violada sobre un recurso que existe, o violación de una restricción `UNIQUE`.
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/v2/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú muestra los módulos del grupo del alumno (más "Clientes", siempre visible) cuando su email figura en el roster del curso; si no figura, muestra todos. Cada listado arranca filtrado por el cliente con el que se inició sesión y permite cambiar a otro titular o a "todos"; la búsqueda, el orden y la paginación de los listados se resuelven en el navegador. **El front es deliberadamente más estricto que la API en varios puntos**: esas reglas están marcadas como "regla de UI" y solo se pueden probar por pantalla — la misma operación enviada directo a la API es aceptada. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
`/v2/ahorros` — Ahorro programado	Tabla con meta, titular, avance y estado; filtros por titular y por estado y búsqueda por nombre de la meta. Ejecuta RF-G5-01.
`/v2/ahorros/new` — Nueva meta de ahorro	Formulario con titular, cuenta de débito, nombre, meta, aporte mensual y tasa, con la proyección de meses y fecha estimada para llegar a la meta. Ejecuta RF-G5-02.
`/v2/ahorros/{id}` — Detalle de la meta	Muestra acumulado, meta, lo que falta, barra de avance y cuenta de débito; mientras el plan está activo permite aportar (RF-G5-06) y editar (RF-G5-04); también eliminar (RF-G5-05).
`/v2/depositos` — Depósitos a plazo	Tabla con número, titular, monto, tasa, vencimiento y estado; filtros por titular y por estado. Ejecuta RF-G5-07.
`/v2/depositos/new` — Constituir depósito a plazo	Formulario con titular, cuenta, monto, plazo y tasa, con la vista previa del interés, la fecha de vencimiento y el total a cobrar. Ejecuta RF-G5-08.
`/v2/depositos/{id}` — Detalle del depósito	Muestra capital, tasa, interés, fechas, días para el vencimiento y estado, y permite cancelar anticipadamente con una confirmación que anticipa el interés estimado (RF-G5-10). Ejecuta RF-G5-09.

- **Regla de UI:** ambos productos solo se ofrecen sobre cuentas activas en guaraníes del titular. La API acepta cualquier cuenta existente, incluso en dólares o de otro cliente.
- **Regla de UI (ahorro):** la meta mínima es de 100.000 guaraníes, el aporte mensual no puede superar la meta, y el plan no puede tardar más de 120 meses en alcanzarla con ese aporte. El nombre de la meta lleva entre 3 y 40 caracteres. La API solo exige valores positivos y un nombre no vacío.
- **Regla de UI (ahorro):** un plan activo con dinero ahorrado no se puede eliminar. La API lo da de baja igual.
- **Regla de UI (depósito):** el monto mínimo es de 1.000.000 de guaraníes y el plazo se elige de una lista (30, 60, 90, 180, 365, 730 o 1095 días). La API acepta cualquier monto positivo y cualquier plazo entero de 30 a 1095.
- **Regla de UI (depósito):** al elegir un plazo la pantalla sugiere una tasa de un tarifario (del 3,5 % a 30 días al 8 % a 1095 días), que el usuario puede editar. La API no tiene tarifario: acepta la tasa que se le envíe.
- El aporte o el depósito por un importe mayor al saldo de la cuenta se rechaza en el navegador antes de llegar a la API.
- Al editar un plan la pantalla envía siempre la tasa, para evitar que la API la reinicie a 0.
- La pantalla avisa cuando un aporte supera lo que falta para la meta, pero lo deja enviar: el plan queda completado con excedente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G5-01 — Listar planes de ahorro

- Al listar sin filtros, la respuesta es 200 y cada elemento incluye `falta_para_meta`.
- Al filtrar por `estado=completado`, todos los elementos devueltos tienen `estado` igual a `completado`.
- Para cada plan devuelto, `falta_para_meta` es igual a `meta_monto` menos `saldo_acumulado`.
- Al enviar `estado=pausado`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G5-02 — Crear un plan de ahorro

- Al crear un plan con meta 1.000.000 y aporte 250.000, la respuesta es 201, `data.estado` es `activo`, `data.saldo_acumulado` es `"0.00"` y `data.falta_para_meta` es `"1000000.00"`.
- Tras crear el plan, el saldo de la cuenta asociada es el mismo que antes.
- Al crear sin `tasaAnual`, la respuesta es 201 y `data.tasa_anual` es `"0.00"`.
- Al crear sin `cuentaId`, o con `metaMonto` igual a 0, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear con una `cuentaId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ningún plan.

RF-G5-03 — Consultar un plan de ahorro

- Dado un plan existente, al consultarlo la respuesta es 200 y `data.id` coincide con el solicitado.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Plan de ahorro no encontrado.".
- Al consultar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G5-04 — Modificar un plan de ahorro

- Dado un plan activo, al cambiar su `metaMonto` la respuesta es 200 y `data.falta_para_meta` refleja la nueva meta.
- Dado un plan con saldo acumulado 500.000, al fijar una meta de 400.000 la respuesta es 409 `CONFLICT` y la meta no cambia.
- Dado un plan `completado` o `cancelado`, al intentar modificarlo la respuesta es 409 `CONFLICT`.
- Dado un plan con `tasa_anual` 6,50, al modificarlo sin enviar `tasaAnual` la respuesta es 200 y `data.tasa_anual` es `"0.00"`.
- El `saldo_acumulado` no cambia tras la modificación.

RF-G5-05 — Dar de baja un plan de ahorro

- Dado un plan sin aportes, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, RF-G5-03 responde 404 y el plan deja de aparecer en RF-G5-01.
- Dado un plan con saldo acumulado, al darlo de baja la respuesta es 204 y el saldo de la cuenta asociada no cambia.
- Dar de baja dos veces el mismo plan devuelve 204 la primera vez y 404 la segunda.

RF-G5-06 — Aportar a un plan de ahorro

- Dado un plan con aporte mensual 250.000 y una cuenta con saldo 1.000.000, al aportar sin cuerpo la respuesta es 201, `data.saldo_acumulado` es `"250000.00"` y `data.cuenta.saldo` es `"750000.00"`.
- Al aportar enviando `monto` 100.000, el saldo acumulado sube exactamente 100.000, sin importar el aporte mensual del plan.
- Tras un aporte, el extracto de la cuenta tiene un movimiento `debito` nuevo con `referencia_tipo` igual a `ahorro` y `referencia_id` igual al id del plan.
- Dado un plan con meta 1.000.000 y saldo 750.000, al aportar 250.000 la respuesta es 201, `data.estado` es `completado` y `data.falta_para_meta` es `"0.00"`.
- Dado un plan al que le faltan 100.000, al aportar 300.000 la respuesta es 201, el plan queda `completado` y `data.falta_para_meta` es `"-200000.00"`.

- Dado un plan `completado`, al aportar la respuesta es 409 `CONFLICT` y la cuenta no se debita.
- Dada una cuenta con saldo menor al aporte, la respuesta es 409 `CONFLICT` con el mensaje "Saldo insuficiente en la cuenta para el aporte." y ni el plan ni la cuenta cambian.
- Dado un plan cuya cuenta está `bloqueada`, al aportar la respuesta es 409 `CONFLICT`.
- Al enviar `monto` igual a 0 o negativo, la respuesta es 400 `VALIDATION_ERROR`.

RF-G5-07 — Listar depósitos a plazo

- Al listar sin filtros, la respuesta es 200 y cada elemento incluye `interes_proyectado` y `dias_restantes`.
- Al filtrar por `estado=activo`, todos los elementos devueltos tienen `estado` igual a `activo`.
- Al filtrar por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- Al enviar `estado=renovado`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G5-08 — Constituir un depósito a plazo

- Dada una cuenta con saldo 5.000.000, al constituir un depósito de 2.000.000 al 10 % por 90 días la respuesta es 201, `data.estado` es `activo`, `data.interes_proyectado` es `"49315.07"`, `data.dias_restantes` es 90 y `data.cuenta.saldo` es `"3000000.00"`.
- La `fecha_inicio` devuelta es la fecha del día y la `fecha_vencimiento` es exactamente `plazoDias` días posterior, ambas en formato `YYYY-MM-DD`.
- Tras constituir, el extracto de la cuenta tiene un movimiento `debito` nuevo con `referencia_tipo` igual a `deposito` y `referencia_id` igual al id del depósito.
- Al constituir con `plazoDias` igual a 29 o 1096, la respuesta es 400 `VALIDATION_ERROR`; con 30 y con 1095 (los límites), es 201.
- Al constituir por un importe mayor al saldo, la respuesta es 409 `CONFLICT` con el mensaje "Saldo insuficiente para constituir el depósito." y el saldo no cambia.
- Al constituir sobre una cuenta `bloqueada`, la respuesta es 409 `CONFLICT`.
- Al constituir sobre una cuenta inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Cuenta no encontrada.".
- Al constituir con `tasaAnual` igual a 0, la respuesta es 201 y `data.interes_proyectado` es `"0.00"`.

RF-G5-09 — Consultar un depósito a plazo

- Dado un depósito recién constituido, al consultarlo la respuesta es 200 y sus datos coinciden con los devueltos al crearlo.
- Dado el depósito sembrado 1 (5.000.000 al 11,5 % por 180 días), `data.interes_proyectado` es `"283561.64"`.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Depósito no encontrado.".
- Al enviar `PUT` o `DELETE` sobre un depósito existente, la respuesta es 405.

RF-G5-10 — Cancelar anticipadamente un depósito

- Dado un depósito constituido hoy por 2.000.000, al cancelarlo la respuesta es 200, `data.estado` es `cancelado`, `data.dias_transcurridos` es 0, `data.interes_generado` es `"0.00"` y el saldo de la cuenta vuelve al valor previo a la constitución.

- Tras cancelar, el extracto de la cuenta tiene un movimiento `credito` nuevo por capital más interés, con `referencia_tipo` igual a `deposito`.
- Dado un depósito activo con fecha de inicio pasada, el `interes_generado` es igual a monto × tasa / 100 × `dias_transcurridos` / 365 redondeado a dos decimales, y la cuenta recibe capital más ese interés.
- Dado un depósito cuyo plazo ya se cumplió, `data.dias_transcurridos` no supera el `plazo_dias` y el interés es igual al proyectado.
- Al cancelar por segunda vez el mismo depósito, la respuesta es 409 `CONFLICT` y la cuenta no vuelve a acreditarse.
- Dados los depósitos sembrados en estado `vencido`, al cancelarlos la respuesta es 409 `CONFLICT`.
- Al cancelar un depósito inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Depósito no encontrado.".

RF-G5-TK — Obtener el token del grupo

- Con el usuario y la contraseña del grupo 5, la respuesta es 200, `data.tokenType` es `Bearer`, `data.expiresIn` es 3600 y `data.grupo` es 5.
- Una request posterior a cualquier endpoint del módulo enviando solo `Authorization: Bearer <token>` (sin `x-api-key`) responde con éxito.
- Con una contraseña incorrecta, la respuesta es 400 `VALIDATION_ERROR` con el mensaje "Usuario o password inválidos."; con un usuario inexistente, la respuesta es idéntica.
- Con las credenciales válidas del grupo 1 sobre esta ruta, la respuesta es 403 `FORBIDDEN`.
- Con un token alterado o vencido en el header `Authorization`, cualquier endpoint responde 401 `UNAUTHORIZED`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre captación de depósitos de ahorro y a plazo	Un depósito a plazo debe instrumentarse con capital, tasa, plazo y fecha de vencimiento ciertos, e informarse al depositante.	Cubierto: el depósito guarda esas condiciones, son inmutables y la API devuelve además el interés proyectado.
Ley 2334/03 de Garantía de Depósitos	Los depósitos del público están cubiertos hasta un tope por el Fondo de Garantía de Depósitos, lo que exige identificar al titular.	No implementado: el sandbox no modela la garantía, y no valida que la cuenta del depósito pertenezca al titular declarado.
Normativa del BCP sobre transparencia de tasas pasivas	La tasa ofrecida debe publicarse y aplicarse según el tarifario vigente de la entidad.	No implementado en la API: acepta cualquier tasa hasta 999,99 %; el tarifario existe solo como sugerencia en la pantalla web.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El cliente debe conocer de antemano las condiciones de cancelación anticipada y el interés que percibirá.	Parcial: la web anticipa el interés estimado antes de confirmar la cancelación; la API no expone una simulación previa.
Buenas prácticas de productos de ahorro programado	El dinero aportado debe poder rescatarse, y el plan debe devengar el interés pactado.	No implementado: no hay rescate del saldo acumulado ni liquidación de intereses, y dar de baja un plan no devuelve el dinero.

Brechas conocidas del sandbox

- El dinero de un plan de ahorro no se puede retirar: no existe rescate ni al completar la meta ni antes. Lo aportado sale de la cuenta y no vuelve.
- `DELETE /ahorros/{id}` no valida nada: da de baja un plan activo con saldo acumulado, y ese saldo queda inaccesible.
- El estado `cancelado` de un plan de ahorro solo aparece en los datos sembrados: no hay endpoint para cancelar un plan.
- La tasa anual del ahorro es decorativa: no se liquidan intereses sobre el saldo acumulado.
- Un aporte puede superar la meta sin recorte; el excedente queda en el plan y `falta_para_meta` pasa a negativo.
- `PUT /ahorros/{id}` reinicia la tasa a 0 si no se envía, y bajar la meta hasta el saldo acumulado no completa el plan.
- El estado `vencido` de un depósito solo aparece en los datos sembrados: la API no vence depósitos ni acredita el capital al vencimiento. Un depósito pasado de fecha sigue `activo` con `dias_restantes` negativo hasta que alguien lo cancela.
- Los depósitos sembrados en `vencido` no tienen salida: no se pueden cancelar ni cobrar por la API.
- No hay penalidad por cancelación anticipada: se cobra el interés completo de los días transcurridos.
- La cancelación acredita la cuenta sin mirar su estado (bloqueada o cerrada), mientras que la constitución sí lo exige.
- Ni el plan ni el depósito validan que la cuenta pertenezca al titular indicado, y el depósito no guarda moneda: uno constituido desde una cuenta en dólares es indistinguible, en sus propios datos, de uno en

guaraníes.

- Si la cuenta asociada a un depósito se da de baja, el depósito queda sin poder cancelarse (404) y el capital inmovilizado.
- No hay control de frecuencia de aportes ni débito automático: el "aporte mensual" es solo el importe por defecto.
- No existe control de titularidad: cualquier credencial del curso puede aportar, constituir o cancelar productos de cualquier cliente.